Documentação
- SQL Avançado -
BD *classicmodels*

IFCE Campus Crato
Disciplina: Banco de Dados II
Professor: Marco André Santos Machado

Equipe:
Carla Mariana Tenório de Albuquerque Sobreira de Oliveira;
Cleverson Alves de Moura;
Gustavo Kauan Oliveira Lemos;
Iruan Gonçalves de Moura Silva;
Mário Ítalo Oliveira Silva.

Repositório do GitHub:
https://github.com/Carla-newDEV/ClassicModels-Data-Analysis.git

Q1 - Panorama Temporal e Ranking (Funções de Janela)

1. Objetivo Geral

O objetivo desta etapa é analisar a evolução do faturamento mensal por cliente, identificando quem são os maiores compradores em cada período e como o volume de compras desses clientes se acumula ao longo do tempo.

- Métrica-Chave: Vendas totais por cliente.
- Segmentação: Agrupamento mensal (Série Temporal).
- Destaque: Ranking "Top 5" clientes por mês.

2. Consulta Analítica: Ranking e Tendência de Vendas

Objetivo: Produzir uma visão que combine o desempenho imediato (ranking mensal) com o histórico do cliente (vendas acumuladas).

Código SQL:

```
WITH vendas_mensais AS (
  SELECT
    DATE_FORMAT(o.orderDate, '%Y-%m') AS ano_mes,
    c.customerName,
    SUM(od.quantityOrdered * od.priceEach) AS total_vendas
  FROM orders o
  JOIN customers c ON o.customerNumber = c.customerNumber
  JOIN orderdetails od ON o.orderNumber = od.orderNumber
  GROUP BY ano_mes, c.customerName
),
ranking AS (
  SELECT
    ano_mes,
    customerName,
    total_vendas,
    -- Ranking por mês
    RANK() OVER (
```

```
    PARTITION BY ano_mes
    ORDER BY total_vendas DESC
  ) AS ranking_mes,
  SUM(total_vendas) OVER (
    PARTITION BY customerName
    ORDER BY ano_mes
  ) AS acumulado_cliente
 FROM vendas_mensais
)
SELECT *
FROM ranking
WHERE ranking_mes <= 5
ORDER BY ano_mes DESC,  rankig_mes;
```

3. Lógica Implementada

- **Série Temporal (ano_mes):** Foi utilizado o DATE_FORMAT na tabela de pedidos para verificar os dados por mês. Essa escolha permite identificar ciclos de compra mensais, que é o padrão chave para análises de faturamento (Closing mensal).
- **Função de Ranking (RANK()):**
    - Partição: O cálculo é reiniciado a cada novo mês (PARTITION BY ano_mes).
    - Tratamento de Empates: Optou-se pela função RANK(). Caso dois clientes possuam exatamente o mesmo valor de compra, ambos receberão a mesma posição (ex: dois "1º lugares"), e a posição seguinte será pulada. Isso é ideal para análise de vendas, onde o mérito do faturamento idêntico deve ser reconhecido.
- **Medida de Tendência (acumulado_cliente):** Foi utilizada uma função agregadora (SUM) com janela. Ela calcula o "Running Total" (Total Acumulado) das compras de cada cliente à medida que os meses avançam, permitindo ver se um cliente que é "Top 1" em um mês específico já possui um histórico sólido de compras ou se foi uma compra isolada.

4. Resultado parcial da query

ano_mes	customerName	total_vendas	ranking_mes	acumulado_cliente
2005-05	Euro+ Shopping Channel	104780.68	1	820689.54
2005-05	Salzburg Collectables	52420.07	2	137480.07
2005-05	Gifts4AllAges.com	50806.85	3	84340.32
2005-05	Souveniers And Things Co.	42251.51	4	133907.12
2005-05	La Rochelle Gifts	41623.44	5	158573.12
2005-04	The Sharp Gifts Warehouse	83984.89	1	143536.27
2005-04	Tekni Collectables Inc.	43525.04	2	81806.55
2005-04	Down Under Souveniers, Inc	41426.81	3	154622.08
2005-04	UK Collectables, Ltd.	37258.94	4	106610.72
2005-04	GiftsForHim.com	36442.34	5	94431.76
2005-03	Mini Gifts Distributors Ltd.	83598.04	1	584188.24
2005-03	Euro+ Shopping Channel	65071.26	2	715908.86
2005-03	Reims Collectables	46656.94	3	126983.19
2005-03	Down Under Souveniers, Inc	33593.32	4	113195.27

Q2 - View Analítica
(monitorar qualidade de dados e priorização)

01. Objetivo Geral

O objetivo dessa etapa é definir um parâmetro de funcionamento de negócios, analisar os dados extraídos do banco e classificar os dados de acordo com os parâmetros definidos.

Parâmetro: Tempo levado entre o pedido do cliente e despacho do produto para envio.

Possíveis Classificações:

- Nenhum Atraso: Tempo menor que 3 dias;
- Médio Atraso: Tempo maior que 3 dias e menor que 5 dias;
- Alto Atraso: Tempo maior que 5 dias e menor que 10 dias;
- Extremo Atraso: Tempo maior que 10 dias;

02. View Análitica: DemoraNoTransporte

Objetivo

Analisar os dados adquiridos através da consulta do banco e classificá-los de acordo com o parâmetro definido.

Código SQL:

```
create or replace view DemoraNoTransporte(NumedoCliente, Cliente, NumeroPedido,
Solicitacao, DiasPassados, Atraso)
as
select c.customerNumber, c.customerName, p.orderNumber, p.orderDate,
coalesce(datediff(p.shippedDate, p.orderDate), 0) as diasPassados,
case
   when coalesce(datediff(p.shippedDate, p.orderDate), 0) >= 10 then "Extremo"
   when coalesce(datediff(p.shippedDate, p.orderDate), 0) >= 5 and
coalesce(datediff(p.shippedDate, p.orderDate), 0) < 10 then "Alto"
   when coalesce(datediff(p.shippedDate, p.orderDate), 0) >= 3 and
coalesce(datediff(p.shippedDate, p.orderDate), 0) < 5 then "Médio"
```

```
  when coalesce(datediff(p.shippedDate, p.orderDate), 0) < 3 then "Nenhum"
end
from customers as c inner join orders as p on c.customerNumber = p.customerNumber
where p.status != "cancelled";
```

Lógica Implementada:

Dias Passados: O tempo em dias é calculado através da função datediff que é capaz de calcular a diferença em dias entre duas datas, no caso as datas:

- orderDate(Dia do Pedido) da tabela orders(Pedidos).
- shippedDate(Dia do Despacho) da tabela orders(Pedidos).

A função coalesce é usada para quando a diferença entre datas retorna null, através da função esse retorno passa a ser zero.

Atraso: É determinado através da função case, com uma série de condicionais que classifica o atraso de acordo como o quantidade de dias passados.

03. View Análitica: DemoraNoTransporteCTE

Objetivo

Diminuir a repetição de código e tornar a interpretação dele mais acessível.

Código SQL:

```
create or replace view DemoraNoTransporteCTE(NumeroCliente, Cliente, NumeroPedido,
Solicitacao, DiasPassados, Atraso)
as
with PedidosCalculados
as (
select c.customerNumber, c.customerName, p.orderNumber, p.orderDate,
coalesce(datediff(p.shippedDate, p.orderDate), 0) as calendarDays
from customers as c inner join orders as p on c.customerNumber = p.customerNumber
where p.status != "cancelled"
)
select
```

```
customerNumber, customerName, orderNumber, orderDate,calendarDays,
case
   when calendarDays >= 10 then "Extremo"
   when calendarDays >= 5 and calendarDays < 10 then "Alto"
   when calendarDays >= 3 and calendarDays < 5 then "Médio"
   when calendarDays < 3 then "Nenhum"
end
from PedidosCalculados;
```

Lógica Implementada:

Uso de CTE: O tempo em dias é calculado agora através da CTE PedidosCalculados, que realiza o cálculo, porém não classifica o tempo. Após isso, o tempo já calculado é consultado de dentro da CTE e o atraso passa a ser classificado utilizando os dados adquiridos.

4. Instruções de Consumo

Para visualizar os dados gerados pelas views, utilize os comandos abaixo:

SQL:

```
-- Para a view sem CTE:
select * from DemoraNoTransporte;

-- Para o view com CTE
select * from DemoraNoTransporteCTE;
```

Q3- Garantia de regra de negócio (trigger com auditoria)

01. Regra de monitoria:

Impedir que dados importantes sejam deletados, e registrar qualquer possível tentativa de tal. No caso,os dados protegidos são referentes aos clientes, aos pagamentos, e aos pedidos,e aos detalhes dos pedidos.

02. Resumo dos triggers e suas funcionalidades:

- O código possui vários triggers justamente porque atua em diferentes tabelas.
- Os triggers possuem a mesma lógica e executam a mesma função em contextos diferentes.
- A ideia central é registrar no log:
  data/hora
  usuário
  operação
  qual registro (ou identificador) tentaram deletar

03. O código na prática:

Vamos pegar o primeiro trigger como exemplo.

DELIMITER $$ */Muda o delimitador padrão do SQL de ; para $$, permitindo que o corpo do trigger contenha múltiplos comandos sem que o MySQL encerre a execução prematuramente/*

CREATE TRIGGER bloqueia_delete_customer */Cria o gatilho (trigger) com o nome especificado para identificar a ação no banco de dados/*

BEFORE DELETE ON customers */Define que o gatilho deve ser disparado imediatamente antes de qualquer tentativa de exclusão na tabela de clientes/*

FOR EACH ROW */Determina que a lógica do trigger será aplicada individualmente para cada linha que o comando DELETE tentar remover/*

BEGIN */Início do bloco de instruções que compõem a lógica do trigger/*

INSERT INTO log_tentativas_deletar (timestamp, user, operation, issue, sql_text) */Indica que os dados da tentativa de exclusão serão gravados na tabela de auditoria, especificando as colunas de tempo, usuário, operação, descrição e o comando SQL/*

VALUES (NOW(), CURRENT_USER(), 'DELETE', ...) */Define os valores a serem inseridos: NOW() captura o momento exato, CURRENT_USER() identifica o autor da tentativa e 'DELETE' registra o tipo de ação/*

CONCAT('Tentativa de excluir cliente: ', OLD.customerNumber) */Usa a função de concatenação para criar uma descrição legível, utilizando o prefixo OLD. para acessar o número do cliente que estava prestes a ser apagado/*

CONCAT('DELETE FROM customers WHERE customerNumber = ', OLD.customerNumber); */Gera e armazena uma representação textual do comando SQL que disparou o trigger para fins de auditoria/*

SIGNAL SQLSTATE '45000' */Comando que força um erro definido pelo usuário, interrompendo imediatamente a operação de DELETE e impedindo que o registro saia do banco/*

SET MESSAGE_TEXT = 'Exclusão não permitida na tabela customers'; */Define a mensagem personalizada que será exibida para o usuário ou aplicação que tentou realizar a exclusão/*

END$$ */Fim do bloco de instruções do trigger e encerramento do comando usando o delimitador personalizado/*

DELIMITER ; */Retorna o delimitador padrão do sistema para o ponto e vírgula (;)/*

Q4 - Automação de relatório (stored procedure)

01. Objetivo Geral

Entregar, por meio de uma Stored Procedure, um Relatório Parametrizado que gere uma visão consolidada de um aspecto relevante do banco de dados.

02. View vw_pedidos_totais

```
CREATE VIEW vw_pedidos_totais AS
SELECT
   o.orderNumber,
   c.country,
   o.orderDate,
   SUM(od.quantityOrdered * od.priceEach) AS total_por_pedido
FROM orders o
JOIN customers c ON o.customerNumber = c.customerNumber
JOIN orderdetails od ON o.orderNumber = od.orderNumber
GROUP BY o.orderNumber, c.country, o.orderDate;
```

Função:

Entrega informações de pedidos(orderNumber e orderDate), clientes(country) e custo total de cada pedido (Cálculo realizado pela *função de agregação* padrão do SQL: SUM(), que soma os resultados da operação realizada dentro dos parêntesis - que usa a quantityOrdered e o priceEach da tabela orderdetails, multiplicando a quantidade de cada produto por seu respectivo preço - gerando, assim, o total_por_pedido).

A principal razão de sua existência é ser a fonte de dados para a Procedure do tópico a seguir.

03. Procedure sp_relatório_vendas

```
DELIMITER $UvU$
```

```
CREATE PROCEDURE sp_relatorio_vendas(
   IN inicio_periodo DATE,
   IN fim_periodo DATE,
   IN nacao VARCHAR(50)
)
BEGIN

   IF inicio_periodo > fim_periodo THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Data de início maior que a do fim.';
   END IF;


   SELECT
      country,
      COUNT(*) AS total_pedidos,
      SUM(total_por_pedido) AS receita_total,
      AVG(total_por_pedido) AS valor_medio_dos_pedidos
   FROM vw_pedidos_totais
   WHERE orderDate BETWEEN inicio_periodo AND fim_periodo
   AND country = nacao
   GROUP BY country;

END$UvU$

DELIMITER ;
```

Função:

Gera um relatório que mostra o valor total de todos os pedidos feitos por clientes de determinado *país* em determinado *período de tempo* e o valor médio de cada pedido(os dados necessários para essas ações vêm dos parâmetros e da vw_pedidos_totais)

Parâmetros:

inicio_periodo - data;
fim_periodo - data;

nacao - país escrito da mesma forma que foi guardado no BD.

As datas têm o formato - ‘ano-mês-dia’ - entre aspas simples;
O país também deve estar entre aspas simples.

Verificação:

O IF é responsável por verificar se o inicio_periodo é maior que o fim_periodo e, caso seja, um erro é lançado e a execução é cessada.

Ex. de chamada da Procedure:

```
CALL sp_relatorio_vendas('2005-05-01', '2005-05-02', 'USA');
```

Saída da chamada de ex.:

country	USA
total_pedidos	112
receita_total	3273280.05
valor_medio_dos_pedidos	29225.714732

Q5 - Views para Diferentes Perfis

01. Objetivo Geral

O objetivo desta etapa é fornecer camadas de abstração de dados que atendam a dois perfis distintos de usuários do sistema de vendas:

> **Perfil Gerencial:** Focado em indicadores de desempenho (KPIs) consolidados, tendências mensais e visão estratégica.
>
> **Perfil Analítico:** Focado na operação diária, permitindo a inspeção detalhada de cada transação, cliente e produto.

02. View Gerencial: vw_painel_vendas_gerencial

Objetivo

Fornecer indicadores de desempenho consolidados para a diretoria, permitindo a análise de tendências de receita e volume de pedidos ao longo do tempo.

Código SQL:

```
CREATE OR REPLACE VIEW vw_painel_vendas_gerencial AS
SELECT
   DATE_FORMAT(o.orderDate, '%Y-%m') AS mes_venda,
   pl.productLine AS linha_produto,
   COUNT(DISTINCT o.orderNumber) AS total_pedidos,
   SUM(od.quantityOrdered * od.priceEach) AS receita_total,
   ROUND(SUM(od.quantityOrdered * od.priceEach) / COUNT(DISTINCT
o.orderNumber), 2) AS ticket_medio
FROM orders o
JOIN orderdetails od ON o.orderNumber = od.orderNumber
JOIN products p ON od.productCode = p.productCode
JOIN productlines pl ON p.productLine = pl.productLine
WHERE o.status != 'Cancelled'
GROUP BY
   DATE_FORMAT(o.orderDate, '%Y-%m'),
```

```
    pl.productLine
ORDER BY
    mes_venda DESC,
    receita_total DESC;
```

Lógica Implementada

Dimensão Temporal: Utiliza a função DATE_FORMAT para extrair o ano e o mês da data do pedido, permitindo agrupamentos mensais.

Métricas de Desempenho: * total_pedidos: Utiliza COUNT(DISTINCT) para contar pedidos únicos, evitando duplicidade devido aos itens do pedido.

- receita_total: Soma o valor total de cada item (quantidade * preço unitário).
- ticket_medio: Razão entre a receita e o número de pedidos, arredondada para duas casas decimais.

Filtro de Integridade: A cláusula WHERE o.status != 'Cancelled' garante que a receita e o volume reflitam apenas vendas efetivadas.

03. View Analítica: vw_painel_vendas_detalhadas

Objetivo

Permitir que analistas operacionais inspecionem dados transacionais diários, cruzando informações pontuais sobre clientes, produtos específicos e status de entrega em uma única interface.

Código SQL:

```
CREATE OR REPLACE VIEW vw_painel_vendas_detalhadas AS
SELECT
    o.orderDate AS data_venda,
    o.orderNumber AS numero_pedido,
    c.customerName AS nome_cliente,
    p.productName AS nome_produto,
    p.productLine AS linha_produto,
    od.quantityOrdered AS quantidade,
```

```
    od.priceEach AS valor_unitario,
    (od.quantityOrdered * od.priceEach) AS valor_total_item,
    o.status AS status_pedido
FROM orders o
JOIN customers c ON o.customerNumber = c.customerNumber
JOIN orderdetails od ON o.orderNumber = od.orderNumber
JOIN products p ON od.productCode = p.productCode;
```

Lógica Implementada

Planificação de Dados: Realiza o JOIN entre quatro tabelas primárias (orders, customers, orderdetails e products), transformando um modelo relacional complexo em uma tabela virtual plana de fácil leitura.

Campo Calculado: Inclui a coluna valor_total_item, que automatiza o cálculo de faturamento por linha, eliminando a necessidade de processamento extra em ferramentas de BI ou Excel.

Rastreabilidade: Expõe o status_pedido, essencial para que o analista identifique gargalos logísticos ou pedidos em processamento.

04. Instruções de Consumo

Para visualizar os dados gerados pelas views, utilize os comandos abaixo:

SQL

```
-- Para o perfil gerencial
SELECT * FROM vw_painel_vendas_gerencial;

-- Para o perfil analítico (exemplo filtrando um cliente específico)
SELECT * FROM vw_painel_vendas_detalhadas WHERE nome_cliente LIKE 'Mini%';
```